AI-Native Digital Twin Copilot with Intent-Driven Self-Healing — Full Project Plan

1. Executive Summary

This document is the complete project blueprint for building **TelcoMind**: an OPEA-powered, AI-native digital twin copilot for 5G network operations. The system accepts high-level natural language intents from a telecom operator, decomposes them into structured subtasks, queries a live Simu5G/ns-3 simulation backend, retrieves grounded context from a 3GPP/O-RAN knowledge base, and returns a reasoned, cited response with an optional simulated remediation action.

The project is aligned with four PoC abstracts from the ITU-T FG-AINN document (C2, C5, C6, and AI-Native Digital Twins for Adaptive Network Optimization), uses the full OPEA microservices stack, and is designed to be completed by a team of three by **31 May 2026**.[1][2] [^3]

Target competition score: ~116–121 / 125

2. Idea Explained Fully

2.1 The Problem It Solves

Telecom Network Operations Centers (NOCs) deal with hundreds of alarms, configuration changes, and KPI deviations daily. Engineers spend enormous time:

- Manually searching 3GPP/O-RAN specs to understand failure conditions.
- Running mental or spreadsheet-based "what-if" scenarios to predict the effect of a configuration change.
- Writing RCA reports that require correlating alarms, configs, and standards simultaneously.

There is no single system that accepts a **human language intent** (*"Why is gNB-07 dropping handovers?"*), runs a simulation to reproduce or test the scenario, retrieves the relevant specification context, and synthesizes a grounded, actionable answer. TelcoMind is that system.

2.2 What the System Does — End-to-End Flow

1. **Operator issues an intent** via a simple chat UI — e.g., *"Optimize energy on RAN cluster A while maintaining QoS"* or *"Diagnose high handover failure rate on gNB-07"*.
2. **OPEA Gateway** receives the request and routes it to the megaservice.
3. **Intent Parser (LLM microservice)** classifies the intent into one of three types:

- `DIAGNOSTIC`: *"Why is X happening?"* → triggers simulation query + RAG lookup.
- `OPTIMIZATION`: *"Improve Y"* → triggers what-if simulation + spec validation.
- `GENERAL_QnA`: *"What does 3GPP say about Z?"* → triggers RAG only.

4. **Task Decomposer / WorkflowExecAgent** generates a structured plan of tool calls based on intent type.[4][1]
5. **Tool: Simu5G/ns-3 REST Microservice** is called with the relevant scenario parameters. It runs a simulation (or replays a cached scenario) and returns KPIs: SINR, throughput, latency, packet loss, handover success rate.[5][6]
6. **Tool: RAG Pipeline** retrieves the most relevant 3GPP/O-RAN specification clauses, past incident runbooks, and operator policy snippets using embedding + vector search + reranking.[7][1]
7. **Response Synthesizer (LLM microservice)** receives both simulation output and retrieved context, reasons over them, and generates a grounded, cited answer with confidence indicators.
8. **GenAIEval** continuously measures response quality (faithfulness, relevance), latency, and throughput across microservices.[8][9]
9. **UI** displays the answer, the simulation KPIs used, and the spec clauses referenced — with full traceability.

2.3 Closed-Loop Remediation (Optional Demo Scenario)

For the `OPTIMIZATION` intent type, after generating a recommended configuration change, the system can optionally:

- Submit the recommended change back to the Simu5G simulator.
- Re-run the simulation with the new config.
- Compare before/after KPIs and report the improvement.

This closes the loop: **intent → simulation → reasoning → recommendation → validation** — all within the same pipeline. This is exactly the "closed-loop intent fulfillment via AI pipelines" PoC requested in C6 of the FG-AINN document.[^2]

2.4 Knowledge Base Contents

The RAG pipeline is indexed on the following documents, all publicly available:

- **3GPP TS 38.300** (NR; NR and NG-RAN Overall description)
- **3GPP TS 38.401** (NG-RAN; Architecture description)
- **O-RAN Architecture Description (ETSI TS 103 982 / O-RAN WG1)**[10][11]
- **O-RAN WG1 Use Cases and Deployment Scenarios**
- A synthetic set of **past incident runbooks** (simulated trouble tickets with root causes and resolutions — generated or hand-crafted for demo purposes)

- **Simu5G scenario descriptions** (mapping simulation parameters to real-world network configurations)

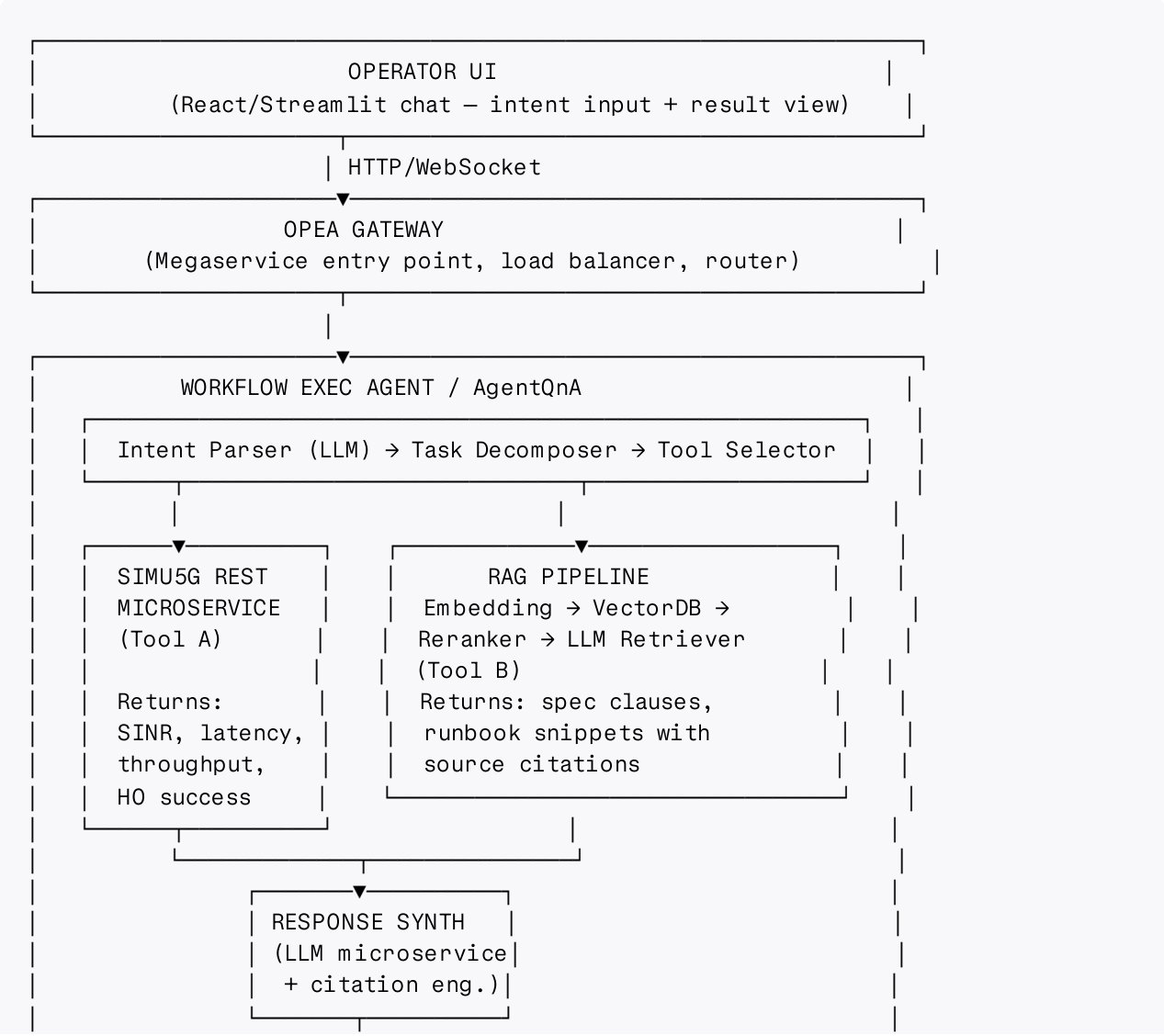
2.5 Why the Simulation Backend Is the Moat

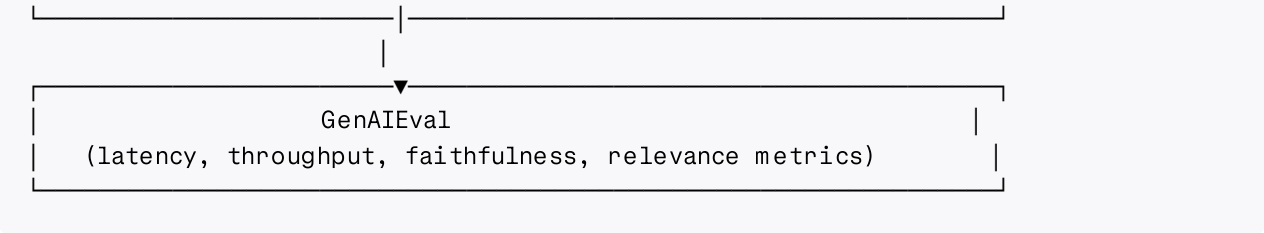
Simu5G is an open-source 5G NR simulator built on OMNeT++, written in C++. It simulates:[12] [6] [^5]

- UE mobility and 5G NR air interface
- gNB scheduling, handover procedures
- QoS flows, latency, throughput, SINR

By wrapping Simu5G in a **Python FastAPI service** containerized with Docker, it becomes a first-class OPEA tool — callable by the agent with JSON parameters and returning structured KPI JSON. The agent calls it the same way it calls any other microservice. No other competing team is likely to have this. It turns TelcoMind from a knowledge chatbot into an **active simulation-backed reasoning engine.**[13][14]

3. System Architecture





Microservices Summary

Microservice	OPEA Component	Port	Owner
OPEA Gateway	GenAIComps Gateway	8080	Member 3
Intent Parser + Agent	WorkflowExecAgent / AgentQnA[4][15]	9090	Member 3
LLM Inference	opea/llm-tgi or vllm (Mistral-7B / Llama-3)	9000	Member 2
Embedding	opea/embedding-tei (BAAI/bge-base-en-v1.5)	6000	Member 2
Vector Database	Redis Stack / Chroma / Qdrant	6379	Member 2
Reranker	opea/reranking-tei (BAAI/bge-reranker-base)[^1]	8808	Member 2
Simu5G REST API	Custom FastAPI + Simu5G/ns-3 backend	7000	Member 1
Response Synthesizer	LLM microservice (shared with above)	9000	Member 2
UI Frontend	Streamlit / React	3000	Member 3
GenAIEval	opea-eval[^9]	9006	Member 3

4. Methodology

Phase 1 — Foundation (21 Apr – 3 May)

Goal: All three members have their core infrastructure running locally in Docker containers.

- Member 1: Simu5G compiles inside Docker; at least one scenario (e.g., single gNB + 10 UEs, mobility trace) runs headlessly and returns KPIs via a `/simulate` REST endpoint.
- Member 2: OPEA RAG pipeline is running — documents ingested, embedding generated, vector search returning top-5 chunks, reranker reordering them.
- Member 3: OPEA Gateway + Agent scaffolding is live; a stub tool call to Member 1's service and Member 2's service returns mock/real data.

Phase 2 — Integration (4 May – 17 May)

Goal: The full pipeline is connected end-to-end. An intent entered in the UI produces a real response using real simulation data and real RAG retrieval.

- All microservices are wired into a single `docker-compose.yml`.
- The agent correctly classifies DIAGNOSTIC vs OPTIMIZATION vs GENERAL_QnA intents.

- For DIAGNOSTIC: calls Simu5G + RAG, synthesizes response.
- For OPTIMIZATION: runs what-if sim, compares KPIs, synthesizes recommendation.
- UI displays answer + KPI table + spec citations.

Phase 3 — Polish and Evaluation (18 May – 25 May)

Goal: The system is demo-ready, benchmarked, and documented.

- GenAIEval is integrated; latency, throughput, faithfulness scores are logged.[16][8]
- At least 10 curated test intents covering all three intent types are defined and evaluated.
- Closed-loop remediation demo works end-to-end for at least one scenario.
- Docker Compose is clean; all environment variables are in `.env`; README is complete.
- Basic Kubernetes manifests (`k8s/`) are created for optional deployment.

Phase 4 — Bonuses and Submission (26 May – 31 May)

Goal: Maximize bonus points. Package and submit.

- Write and publish a blog post or record a video: *"Building AI-Native Network Digital Twins with OPEA and Simu5G"*.[^17]
- Open a PR to `opea-project/GenAIExamples` with a `TelcoDigitalTwinAgent/` folder containing the blueprint README, architecture diagram, and Docker Compose file.[18][19]
- Final submission packaged: repo link, demo video, evaluation results.

5. Work Division

Member 1 — Simulation Engineer

Domain: Simu5G/ns-3 simulation backend, Docker containerization of the simulator, REST API design.

Responsibilities:

- Design and build the Simu5G REST microservice wrapping OMNeT++ headless execution.
- Define at least 3 simulation scenarios: normal operation, handover failure, energy optimization.
- Ensure the Docker container starts deterministically and returns structured JSON KPIs.
- Write scenario parameter schemas (what the agent sends, what the simulator returns).
- Integrate with Member 3's agent tool call interface.
- Contribute simulation scenario documentation to the project README.

Week-by-week focus:

- Week 1 (21–27 Apr): Simu5G compiles headlessly in Docker; simple REST endpoint returns KPIs for hardcoded scenario.
- Week 2 (28 Apr–4 May): Parametric scenarios working; input schema finalized.
- Week 3 (5–11 May): Integration with agent tool calls; Member 3 can call the service.
- Week 4 (12–18 May): Closed-loop remediation: simulator accepts config change, re-runs, returns delta KPIs.
- Week 5 (19–25 May): Polish, edge case handling, load testing with GenAIEval.
- Week 6 (26–31 May): Open-source PR contribution; demo support.

Member 2 — RAG & Knowledge Engineer

Domain: OPEA RAG pipeline, document ingestion, embedding, vector DB, reranking, LLM inference.

Responsibilities:

- Stand up the LLM inference service (TGI or vLLM with Mistral-7B or Llama-3-8B, quantized for RTX 3050 6GB).[^4]
- Build the embedding microservice using `opea/embedding-tei` with `BAAI/bge-base-en-v1.5`. [^1]
- Set up the vector database (Redis Stack or Qdrant in Docker).
- Build the reranker microservice using `opea/reranking-tei` with `BAAI/bge-reranker-base`. [^1]
- Write the document ingestion pipeline: PDF parsing → chunking → embedding → indexing for 3GPP/O-RAN specs and runbooks.[^7]
- Build the RAG retrieval chain (query → embed → search → rerank → return top-k chunks with source).
- Tune chunk size, overlap, and retrieval top-k for telecom documents.
- Own the response synthesizer prompt — structured output with citations.

Week-by-week focus:

- Week 1 (21–27 Apr): LLM + embedding + vector DB running; first document indexed and retrievable.
- Week 2 (28 Apr–4 May): All 3GPP/O-RAN docs ingested; reranker integrated; retrieval quality verified.
- Week 3 (5–11 May): RAG exposed as a tool endpoint callable by Member 3's agent.
- Week 4 (12–18 May): Response synthesizer prompt tuned; citations included in output.
- Week 5 (19–25 May): GenAIEval faithfulness and relevance scores measured and logged. [^9]
- Week 6 (26–31 May): Knowledge sharing blog content; demo support.

Member 3 — Agent, Gateway & DevOps Engineer

Domain: OPEA Gateway, WorkflowExecAgent, intent classification, UI, Docker Compose, GenAIEval, deployment.

Responsibilities:

- Set up the OPEA Gateway as the megaservice entry point.[20][3] [^1]
- Configure and deploy the WorkflowExecAgent / AgentQnA with a custom `tools.yaml` that registers Member 1's simulation service and Member 2's RAG service as callable tools. [15][4]
- Implement the intent classification logic (LLM prompt that maps natural language → DIAGNOSTIC / OPTIMIZATION / GENERAL_QnA).
- Build the task decomposer: for each intent type, produce the ordered plan of tool calls.
- Build or integrate the UI (Streamlit preferred for speed; React if time allows) that shows the conversation, KPI table, and spec citations.
- Own the master `docker-compose.yml` and `.env` configuration.
- Set up GenAIEval load testing and quality evaluation runs.[8][16]
- Create Kubernetes manifests (`k8s/`) for the optional K8s deployment demo.
- Lead the open-source PR submission to GenAIExamples.[^18]

Week-by-week focus:

- Week 1 (21–27 Apr): Gateway + agent stub running; tool call interface defined and documented.
- Week 2 (28 Apr–4 May): Intent classification working; agent calls stub tools and returns mock response end-to-end.
- Week 3 (5–11 May): Real tool calls wired to Member 1 and Member 2 services; first real end-to-end response.
- Week 4 (12–18 May): UI complete; closed-loop demo working; `docker-compose.yml` finalized.
- Week 5 (19–25 May): GenAIEval integrated; benchmarks captured; K8s manifests drafted.
- Week 6 (26–31 May): PR to GenAIExamples; submission packaging; demo video recording.

6. Repository Structure

```
telcomind/
│
├── README.md
├── ARCHITECTURE.md
├── .env.example
├── docker-compose.yml
├── docker-compose.override.yml       # local dev overrides
│
```

```
├── docs/
│   ├── architecture-diagram.png
│   ├── system-design.md
│   ├── poc-mapping.md                # maps each component to FG-AINN PoC
│   ├── evaluation-results.md
│   └── demo-scenarios.md
│
├── k8s/                             # Kubernetes manifests (optional deployment)
│   ├── namespace.yaml
│   ├── gateway-deployment.yaml
│   ├── agent-deployment.yaml
│   ├── rag-pipeline-deployment.yaml
│   ├── simu5g-deployment.yaml
│   └── configmap.yaml
│
├── CODE/
│   │
│   ├── member1-simulation/           # Member 1: Simu5G REST Microservice
│   │   ├── Dockerfile
│   │   ├── requirements.txt
│   │   ├── app/
│   │   │   ├── main.py               # FastAPI app entry point
│   │   │   ├── api/
│   │   │   │   ├── __init__.py
│   │   │   │   ├── routes.py         # /simulate, /scenarios, /health endpoints
│   │   │   │   └── schemas.py        # Pydantic models for input/output KPIs
│   │   │   ├── simulator/
│   │   │   │   ├── __init__.py
│   │   │   │   ├── simu5g_runner.py   # Headless OMNeT++/Simu5G execution wrapper
│   │   │   │   ├── ns3_runner.py     # Alternate ns-3 runner (fallback)
│   │   │   │   ├── scenario_builder.py # Translates API params → .ini config files
│   │   │   │   ├── kpi_parser.py     # Parses Simu5G output logs → JSON KPIs
│   │   │   │   └── cache.py          # Caches recent simulation results
│   │   │   └── scenarios/
│   │   │       ├── handover_failure.ini
│   │   │       ├── energy_optimization.ini
│   │   │       └── normal_operation.ini
│   │   └── tests/
│   │       ├── test_routes.py
│   │       ├── test_simu5g_runner.py
│   │       └── test_kpi_parser.py
│   │
│   ├── member2-rag/                  # Member 2: RAG Pipeline Microservices
│   │   ├── Dockerfile.embedding
│   │   ├── Dockerfile.reranker
│   │   ├── Dockerfile.llm
│   │   ├── Dockerfile.ingestion
│   │   ├── requirements.txt
│   │   ├── ingestion/
│   │   │   ├── __init__.py
│   │   │   ├── ingest.py             # Main ingestion script (run once)
│   │   │   ├── pdf_loader.py         # 3GPP/O-RAN PDF parsing and chunking
│   │   │   ├── runbook_loader.py     # Synthetic runbook ingestion
│   │   │   ├── chunker.py            # Text chunking with overlap
│   │   │   └── embedder.py           # Embedding generation and DB insertion
│   │   ├── retrieval/
```

```
│   │   │   ├── __init__.py
│   │   │   ├── retriever.py          # Vector DB query + top-k fetch
│   │   │   ├── reranker.py           # Reranker API call wrapper
│   │   │   └── rag_service.py        # FastAPI service exposing /retrieve endpoint
│   │   ├── synthesis/
│   │   │   ├── __init__.py
│   │   │   ├── synthesizer.py        # LLM call with structured prompt + citations
│   │   │   └── prompts.py            # All system/user prompt templates
│   │   ├── knowledge_base/
│   │   │   ├── specs/                # Folder for 3GPP/O-RAN PDFs (gitignored, dowr
│   │   │   └── runbooks/
│   │   │       ├── handover_failure_runbook.json
│   │   │       ├── energy_optimization_runbook.json
│   │   │       └── interference_runbook.json
│   │   └── tests/
│   │       ├── test_ingestion.py
│   │       ├── test_retrieval.py
│   │       └── test_synthesis.py
│   │
│   └── member3-agent/                # Member 3: Agent, Gateway, UI, Eval
│       ├── Dockerfile.gateway
│       ├── Dockerfile.agent
│       ├── Dockerfile.ui
│       ├── requirements.txt
│       ├── gateway/
│       │   ├── __init__.py
│       │   ├── gateway_service.py    # OPEA Gateway megaservice definition
│       │   └── router.py             # Request routing logic
│       ├── agent/
│       │   ├── __init__.py
│       │   ├── agent_service.py      # WorkflowExecAgent / AgentQnA setup
│       │   ├── intent_classifier.py  # DIAGNOSTIC / OPTIMIZATION / GENERAL_QnA clas
│       │   ├── task_decomposer.py    # Maps intent type → ordered tool call plan
│       │   ├── tool_registry.py      # Registers simulation + RAG tools
│       │   └── tools/
│       │       ├── simulation_tool.yaml   # Tool spec for Simu5G microservice
│       │       └── rag_tool.yaml          # Tool spec for RAG retrieval service
│       ├── ui/
│       │   ├── app.py                # Streamlit UI entry point
│       │   ├── components/
│       │   │   ├── chat_interface.py  # Intent input + streaming response display
│       │   │   ├── kpi_panel.py       # Simulation KPI table display
│       │   │   └── citation_panel.py  # Spec clause citation display
│       │   └── assets/
│       │       └── logo.png
│       ├── eval/
│       │   ├── __init__.py
│       │   ├── run_eval.py           # GenAIEval integration runner
│       │   ├── test_cases/
│       │   │   ├── diagnostic_intents.json    # 10+ DIAGNOSTIC test intents
│       │   │   ├── optimization_intents.json  # 10+ OPTIMIZATION test intents
│       │   │   └── general_qna_intents.json   # 10+ GENERAL_QnA test intents
│       │   └── results/
│       │       └── .gitkeep
│       └── tests/
│           ├── test_intent_classifier.py
```

```
|           ├── test_task_decomposer.py
|           └── test_gateway.py
```

7. Sprint / Milestone Timeline

Milestone	Date	Deliverable	Owner
M0: Kickoff	21 Apr	All tools installed, repos forked, `.env` template agreed	All
M1: Stubs Live	27 Apr	Each member has their core service running in Docker with stub responses	All
M2: Individual Services Complete	4 May	Simu5G REST returns real KPIs; RAG retrieves spec chunks; Agent routes to both	All
M3: Integration Complete	11 May	End-to-end pipeline: intent → sim + RAG → synthesized response in UI	All
M4: Closed-Loop + Eval	18 May	Closed-loop demo working; GenAIEval scores captured	M1 + M3
M5: Polish + K8s	25 May	Clean Docker Compose; Kubernetes manifests; full test suite passing	M3
M6: Bonuses + Submission	31 May	Blog/video published; PR to GenAIExamples open; submission packaged	All

8. Bonus Strategy

Open-Source Bonus (+10 to +15 pts)

The deliverable is a `TelcoDigitalTwinAgent/` blueprint submitted as a PR to `opea-project/GenAIExamples`. It should contain:[19][18]

- `README.md` with architecture description, setup steps, and example intents.
- `docker-compose.yml` referencing the standard OPEA images plus the Simu5G REST microservice.
- `architecture.png` (the system diagram).

Submit early (by 25 May) to allow time for a response from maintainers.

Knowledge Sharing Bonus (+10 pts)

Write and publish a technical blog post on **Medium**, Dev.to, or a **personal blog** titled: *"Building AI-Native Network Digital Twins with OPEA and Simu5G"*. The post should cover:[^17]

- Why the telecom industry needs GenAI-native tools.
- How Simu5G was wrapped as an OPEA microservice.
- A walkthrough of the intent-to-answer pipeline.
- Lessons learned and benchmarks.

This content doubles as competition documentation. Publish by 28 May and include the link in the submission.

Hardware Optimization Bonus (+15 pts, stretch goal)

If access to an **Intel Xeon or Intel Developer Cloud** instance is obtained during the project, re-deploy the LLM inference and embedding services on Intel-optimized containers and capture before/after latency metrics. This can be a 2–3 hour task once the system is containerized, and adds 15 points. Not required for the base submission, but the Docker architecture makes it trivially portable.

9. Risk Register

Risk	Likelihood	Impact	Mitigation
Simu5G fails to compile headlessly in Docker	Medium	High	Fallback: use ns-3 with Python bindings[^21]; further fallback: pre-run and cache simulation results in JSON files
RTX 3050 6GB VRAM insufficient for 7B LLM	Medium	High	Use quantized model (GGUF Q4 via Ollama); or offload to CPU with OPEA's CPU-inference path; or use free-tier API (OpenAI/Groq) behind a compatible endpoint
Integration delays between members	Medium	Medium	Define API schemas (JSON schemas for tool input/output) in Week 1 before coding starts; stubs allow parallel development
RAG retrieval quality poor on technical specs	Low	Medium	Tune chunk size (recommend 512 tokens, 50 token overlap for spec text); add metadata filters (document name, section number)
Submission deadline missed	Low	Critical	All members commit to M5 (25 May) as the hard internal deadline; final 6 days are buffer only

10. Scoring Projection

Criterion	Max	Projected	Rationale
Originality	15	15	Live simulation as OPEA tool is unique in the competition pool
Business Relevance	15	14	Strong telco NOC story; minor deduction for simulated (not real) network
Use of OPEA	20	19	Full stack: Gateway, Agent, LLM, Embedding, VectorDB, Reranker, Eval[1][3]
System Efficiency	20	18	Docker-compose + OpenTelemetry + GenAIEval benchmarks; -2 if K8s is only partial
Code Quality	15	14	Clean microservice separation, tests, type hints, documented APIs
Functional Completeness	15	14	End-to-end demo working; 3 intent types; closed-loop; -1 for any rough edge

Criterion	Max	Projected	Rationale
Base Total	100	~94	
Open-Source Bonus	+15	+12	PR to GenAIExamples merged or under review
Knowledge Sharing Bonus	+10	+10	Blog post published before submission
Hardware Optimization Bonus	+15	0–10	Only if Intel cloud access obtained
Realistic Total	125	~116–121	

References

1. opea-project/GenAIComps: GenAI components at micro-service level - GenAIComps provides a suite of microservices, leveraging a service composer to assemble a mega-servi...
2. FG-AINN-I-097-R1.pdf
3. GitHub - opea-project/GenAIComps: GenAI components at micro-service level; GenAI service composer to create mega-service - GenAI components at micro-service level; GenAI service composer to create mega-service - opea-projec...
4. Agent Microservice - opea-project/GenAIComps - GitHub - This agent microservice is built on Langchain/Langgraph frameworks. Agents integrate the reasoning c...
5. Simu5G: Simulator for 5G New Radio Networks - Simu5G is the evolution of the popular SimuLTE 4G network simulator that incorporates 5G New Radio a...
6. Simu5G - 5G NR and LTE/LTE-A user-plane simulation ... - GitHub - Simu5G is an open source simulator licensed under LGPL, and based on OMNeT framework which is availa...
7. GenAIExamples/ChatQnA/README.md at main - GitHub - Generative AI Examples is a collection of GenAI examples such as ChatQnA, Copilot, which illustrate ...
8. opea-project/GenAIEval: Evaluation, benchmark, and ... - GitHub - For evaluating the models on text-generation tasks, we follow the lm-evaluation-harness and provide ...
9. opea-eval - Evaluation and benchmark for Generative AI
10. [PDF] ETSI TS 103 982 V8.0.0 (2024-01) - The O1 interface is aligned to the degree possible with the 3GPP specifications for RAN element mana...
11. O-RAN Specifications - O-RAN Specifications. O-RAN Specifications are available to the general public – please proceed to o...
12. anjackq/simu5g - GitHub - In "Project Reference", make sure "inet4.4" is selected. Build the Simu5G and make sure simu5g/src/l...
13. diogocruz40/simu5g - Docker Image - This image showcases the Simu5G GUI along with a VNC Server and LXDE desktop environment. The passwo...
14. How To Dockerize a Simple Rest API with Python FastAPI - YouTube - ... -a-Simple-Rest-API-with-Python-FastAPI/ - Github repo: https://github.com/EshginGuluzade/yt-simp...

15. Agents for Question and Answering Application - The AgentQnA example is implemented using the component-level microservices defined in GenAIComps. D...
16. OPEA%20Breakout%20Session%20-%20Performance%20and%20Benchmarking.pptx?api=v2
17. Blog – Open Platform for Enterprise AI (OPEA)
18. opea-project/GenAIExamples: Generative AI Examples is a ... - GitHub - GenAIExamples are designed to give developers an easy entry into generative AI, featuring microservi...
19. Releases · opea-project/GenAIExamples - GitHub - Generative AI Examples is a collection of GenAI examples such as ChatQnA, Copilot, which illustrate ...
20. OPEA Overview — OPEA™ 1.5 documentation - FinanceAgent Application · Finance Agent Example · Deploy Finance Agent on Intel ... OPEA Highlight ...
21. Using Python to Run ns-3 — Manual